CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 15: Test Generation Quality

> **Exam objective:** Improve automated test generation quality by providing existing test files as context, defining fixture conventions, and specifying criteria that distinguish meaningful behavioral tests from trivial assertions.

Q1.

A team asks Claude Code to generate tests for a new service, but the output ignores the project's established test style and helper patterns. What is the best improvement?

A. Provide representative existing test files as context before asking for new tests B. Ask for "better tests" without examples C. Remove all existing tests so the model is not biased D. Focus only on increasing the number of assertions

Answer: A **Explanation:** Existing test files show the project's real conventions for structure, helpers, fixtures, naming, and assertion style. That concrete context helps the model generate tests that fit the codebase. Vague quality requests and assertion-count goals do not anchor style effectively.

Q2.

Claude Code generates tests that instantiate data in ad hoc ways instead of using the project's shared fixtures. Which prompt addition is most helpful?

A. "Use fixtures when appropriate." B. "Follow the fixture convention shown in `userFactory` and `buildOrderFixture`; new tests should reuse those helpers unless a case explicitly requires custom setup." C. "Be more maintainable." D. "Write shorter tests."

Answer: B **Explanation:** The best instruction names the actual fixture helpers and states when to use them. That concrete direction makes the generated tests align with project conventions and improves maintainability. Generic advice to use fixtures is less actionable.

Q3.

A generated test for an HTTP handler only checks that `response.status` exists. Why is this considered low quality?

A. Because status properties should never be asserted B. Because the assertion is trivial and does not verify meaningful behavior or expected outcomes C. Because all HTTP tests must hit a live server D. Because short assertions are always wrong

Answer: B **Explanation:** Meaningful tests validate observable behavior, such as correct status codes, body content, side effects, or error handling. Merely asserting that a property exists is

usually trivial and unlikely to catch regressions. Test quality is about behavioral signal, not assertion length.

Q4.

When asking Claude Code to generate tests for a parser, which instruction best distinguishes meaningful tests from trivial ones?

A. "Write lots of tests." B. "Prefer tests that verify parsed output and error behavior for realistic inputs, not assertions that internal helper functions were called." C. "Use as many mocks as possible." D. "Keep each test under three lines."

Answer: B **Explanation:** This instruction explicitly prioritizes observable behavior over implementation details. That steers generation toward tests that remain valuable even if internals change. More tests, more mocks, or shorter tests do not necessarily improve quality.

Q5.

A codebase has snapshot tests, table-driven tests, and factory helpers. How should that influence automated test generation?

A. The model should ignore those patterns to avoid copying B. Provide those existing tests as context so new tests match proven conventions where appropriate C. Replace all generated tests with snapshots D. Avoid using any helpers to keep tests independent

Answer: B **Explanation:** Existing patterns communicate what the project considers idiomatic and maintainable. Supplying representative examples helps the model choose the right structure for the new tests. It should not blindly convert everything to snapshots or ignore established helpers.

Q6.

A team complains that AI-generated tests are brittle because they assert exact log messages and private method calls. Which guidance is best?

A. "Prefer behavioral assertions on public outcomes and stable contracts; avoid coupling tests to private implementation details unless explicitly required." B. "Assert everything the code does." C. "Increase fixture complexity." D. "Use more snapshots."

Answer: A **Explanation:** High-quality tests focus on stable, externally visible behavior rather than fragile internals. Over-coupling to logs and private methods creates noisy failures when implementation changes without breaking behavior. The guidance in A directly addresses that quality problem.

Q7.

Why do existing test files provide better context than only the production code when generating new tests?

A. They reveal how the team expresses expectations, sets up fixtures, names cases, and balances mocks versus integration-style checks B. They eliminate the need to read production code C. They guarantee zero hallucinations D. They make all generated tests pass automatically

Answer: A **Explanation:** Production code shows behavior targets, but existing tests show how this team prefers to validate those targets. That includes fixture setup, helper use, naming conventions, and assertion idioms. Existing tests do not replace production context or guarantee flawless output.

Q8.

Claude Code generated ten tests for a validator, but eight merely assert that the function returns "not null." What should you change in the instructions?

A. Ask for more lines of code per test B. Specify that each test should verify a distinct rule, edge case, or user-visible outcome rather than existence of any return value C. Ban all assertions except equality D. Remove examples to encourage originality

Answer: B **Explanation:** The problem is lack of behavioral specificity. Instructing the model to cover distinct rules and observable outcomes shifts the focus from trivial existence checks to meaningful validation. More code or fewer examples would not address the core defect.

Q9.

A repository uses a convention where database tests must use transaction-scoped fixtures and rollback automatically. What is the best way to improve generated test quality?

A. Mention that database tests exist B. Provide existing database test examples and explicitly state the fixture and rollback convention C. Ask the model to write simpler tests that avoid the database entirely D. Let the model invent its own fixture pattern

Answer: B **Explanation:** Quality improves when the model can see and follow the exact project pattern for database setup and cleanup. Examples plus explicit conventions reduce inconsistent or unsafe test scaffolding. Leaving the pattern implicit often leads to poor alignment.

Q10.

Which generated test is the strongest example of meaningful behavioral coverage for a rate limiter?

A. A test that asserts the limiter object is defined B. A test that checks a private counter increments from 0 to 1 C. A test that verifies the first N requests succeed and the N+1 request is rejected with the expected public error/result D. A test that snapshots the entire internal state after construction

Answer: C **Explanation:** Option C validates the externally observable contract of the rate limiter under realistic conditions. The other options focus on trivial existence or internal state, which is less robust and less indicative of actual behavior. Behavioral coverage best captures user-facing correctness.

Q11.

A team wants tests generated for a serializer that already has strong fixture builders in `tests\helpers`. Which instruction is best?

A. "Do not use helpers; build raw objects manually for realism." B. "Reuse the existing fixture builders in `tests\helpers` so generated tests match current setup conventions and minimize duplication." C. "Write only one large integration test." D. "Prefer random data in every test."

Answer: B **Explanation:** Reusing established helpers improves consistency, readability, and maintainability, especially when the codebase already standardized setup patterns. Manual raw-object construction often duplicates boilerplate and diverges from project norms. Random data can reduce determinism if used carelessly.

Q12.

What is the main reason to define fixture conventions explicitly in the prompt when generating tests?

A. So the model knows how test data should be created and shared in a way that matches the codebase's maintenance expectations B. So the model stops reading source files C. So tests become longer D. So every test uses the same assertions

Answer: A **Explanation:** Explicit fixture conventions guide the model toward maintainable, idiomatic setup patterns. This reduces ad hoc scaffolding and helps new tests fit naturally into the suite. It does not remove the need to understand source behavior or force uniform assertions.

Q13.

A generated unit test mocks every collaborator and only verifies invocation counts, but the project's best tests usually validate returned values and persisted state. How should you refine the request?

A. Ask for more mocks to improve isolation B. State that tests should prioritize validating outputs and side effects aligned with existing examples, using mocks only where they support behavior-focused assertions C. Remove all collaborators from production code first D. Request a larger number of tiny tests

Answer: B **Explanation:** The refinement should redirect test generation from implementation-centric checks toward observable behavior, while still allowing mocks where they meaningfully support the test. This mirrors how mature test suites balance isolation and value. More mocks alone would worsen the issue.

Q14.

Why is "assert meaningful behavior, not trivial facts" an important instruction for automated test generation?

A. Because AI systems naturally prefer behavior over syntax B. Because trivial facts rarely detect regressions, while meaningful behavior assertions validate real contracts and use cases C. Because trivial assertions are invalid syntax D. Because behavioral tests never require fixtures

Answer: B **Explanation:** Tests exist to catch regressions in important behavior. Assertions about trivial facts like object existence or property presence often provide little signal. Meaningful behavioral checks validate what actually matters to users or dependent components.

Q15.

You want Claude Code to generate tests for a retry mechanism. Which prompt package is most likely to produce high-quality results?

A. Production code only, plus "write tests" B. Existing retry-related tests, fixture/helper conventions, and explicit criteria such as verifying retry counts, stop conditions, and externally visible failure/success outcomes C. A request for 100% line coverage with no other context D. An instruction to avoid assertions about outcomes so tests stay flexible

Answer: B **Explanation:** High-quality test generation comes from combining behavioral intent with project-specific examples and fixture conventions. That gives the model both the "what" and the "how" of good tests in this codebase. Coverage-only goals or outcome-free prompts tend to produce shallow tests.